📘 ملخص Web Programming

CET214 · المحاضرات 01 · 02 · 03

د. أحمد سعيد — ملخص كامل بالعربي. المصطلحات التقنية والكود سايبهم بالإنجليزي زي السلايدز بالظبط عشان الامتحان.

الفهرس

1. المحاضرة 01 — مقدمة عن الإنترنت والويب
2. المحاضرة 02 — أساسيات HTML5
3. المحاضرة 03 — Forms و Semantic HTML و CSS
4. 📋 الجدول الشامل لكل التاجات
5. 📋 الجدول الشامل لكل الـ Attributes
6. 🎨 جداول CSS: الـ Selectors والـ Properties
7. ⚖️ حاجات بتتلخبط فيها (مقارنات سريعة)
8. ⚠️ أشهر الأخطاء — Checklist قبل ما تسلّم

المحاضرة 01 — Introduction to the Internet & Web

الفكرة الكبيرة في سطرين: الإنترنت هو البنية التحتية (شبكة الشبكات). الويب خدمة شغالة **فوق** الإنترنت وبتستخدم HTTP عشان تجيب صفحات HTML. لما تكتب URL بيحصل بالترتيب: **DNS** → **TCP** → **TLS** → **HTTP** → **Render**.

1.1 يعني إيه Internet؟

- **A network of networks**: عشرات الآلاف من الشبكات المستقلة (ISPs، جامعات، شركات، حكومات) متوصلة ببعض وبتربط مليارات الأجهزة.
- الأجهزة بتتكلم مع بعض بقواعد متفق عليها اسمها **Protocols**.
- كل جهاز بيتوصله عن طريق **IP address**:
  - **IPv4** زي `203.0.113.10`
  - **IPv6** زي `2001:db8::1`
  - `192.168.0.1` ده عنوان **private** بتاع شبكة البيت. السيرفرات العامة ليها عناوين **public**.
- البروتوكولات متبنية طبقات فوق بعض: `IP → TCP/UDP → HTTP/SMTP/SSH/DNS…`

1.2 الـ Client–Server Model

```
Browser (Client)              DNS Server                  Web Server
      |--- What is the IP for example.com? --->|
      |<------------ 203.0.113.10 -------------|
      |------------------ HTTP GET /index.html ------------------>|
      |<------------------ 200 OK + HTML content ------------------|
  Render the page
```

الطرف	دوره
Client	المتصفح (Chrome, Firefox, Safari) — بيطلب الصفحة ويعرضها
Server	كمبيوتر بيخزن ويعالج وبيبعت صفحات الويب
DNS	بيترجم اسم الدومين (`example.com`) لـ IP address

> 💡 تشبيه: الـ DNS زي دليل التليفونات — إنت عارف اسم المحل، والدليل بيديك رقمه.

1.3 طبقات البروتوكولات (TCP/IP Model)

الطبقة (من فوق لتحت)	أمثلة	بتعمل إيه
🌐 **Application**	HTTP, FTP, SMTP, DNS	البروتوكولات اللي اليوزر بيتعامل معاها (HTTP للويب، SMTP للإيميل)

الطبقة (من فوق لتحت)	أمثلة	بتعمل إيه
📦 **Transport**	TCP, UDP	**TCP**: توصيل مضمون ومترتب. **UDP**: خفيف وسريع (HTTP/3 شغال على QUIC فوق UDP)
🔀 **Internet**	IP, ICMP	العناوين (IP addresses) والـ routing
🔌 **Network Access**	Ethernet, Wi-Fi	النقل الفيزيائي للداتا (كابلات، واي فاي)

1.4 إزاي الويب شغال (المكونات)

- **IP addresses**: أرقام بتعرّف كل جهاز.
- **DNS**: بيحوّل `google.com` لـ `142.251.208.238`.
- **URL**: بيحدد أنهي resource عايز تجيبه، زي `https://example.com/about?lang=en`.
- **Web server**: بيبعت الصفحات للمتصفح على HTTP/HTTPS.
- **Web browser**: بيطلب الصفحات ويستقبلها ويعرضها.

1.5 تشريح الـ URL (Anatomy of a URL)

```
https :// example.com :8080 /docs/intro.html ?lang=en&page=2 #setup
scheme       host      port       path            query       fragment
```

الجزء	في المثال	بيجاوب على سؤال	ملاحظات
scheme	`https`	بنتكلم بأنهي قواعد؟	`https`، `http`، `mailto:`، `file:`
host	`example.com`	أنهي جهاز؟	الـ DNS بيحوله لـ IP
port	`8080`	أنهي "باب" على الجهاز؟	غالباً مش بيتكتب: **80** لـ http و **443** لـ https
path	`/docs/intro.html`	أنهي resource على السيرفر؟	شكله فولدرات، بس السيرفر هو اللي بيقرر معناه
query	`?lang=en&page=2`	فيه parameters زيادة؟	أزواج `key=value` مفصولة بـ `&`
fragment	`#setup`	أنهي جزء في الصفحة؟	بيعمل scroll للعنصر اللي `id="setup"`

⭐ **نقطة امتحان:** الـ fragment (`#setup`) **عمره ما بيتبعت للسيرفر** — المتصفح بيحتفظ بيه ويروح للعنصر بعد ما الصفحة تحمل، عشان كده مش بيظهر في logs السيرفر.

1.6 مين اللي "بيدير" الإنترنت؟

مفيش جهة واحدة بتتحكم في الإنترنت. فيه منظمات بتحافظ على البنية:

المنظمة	مسؤولة عن إيه	احفظها كده
ICANN	أسماء الدومينات وتوزيع الـ IP addresses (الـ DNS root)	أسماء وأرقام
IETF	ستاندردات بروتوكولات الإنترنت في صورة **RFCs** (TCP/IP, HTTP, TLS)	البروتوكولات

المنظمة	مسؤولة عن إيه	احفظها كده
W3C	ستاندردات الويب (CSS، الـ Web APIs، accessibility/WCAG كتير)	CSS والـ accessibility
WHATWG	الـ **HTML Living Standard** والـ DOM (شركات المتصفحات)	HTML

1.7 تاريخ الويب باختصار

السنة	الحدث
1969	**ARPANET** — أول packet-switched network (جد الإنترنت)
1983	ARPANET تتحول لـ **TCP/IP** ← ميلاد الإنترنت الحديث
1989	**Tim Berners-Lee** يقترح الـ World Wide Web في **CERN**
1991	أول موقع (`info.cern.ch`) يتفتح للناس
1993	**Mosaic** أول متصفح graphical مشهور + CERN تخلي الويب public domain
1994	تأسيس **W3C** + نزول Netscape Navigator
1995	Internet Explorer + اختراع **JavaScript**
2004	عصر **Web 2.0** (Facebook، محتوى من اليوزرز) + Firefox 1.0 + انتشار AJAX
2008	نزول Google Chrome
2010s	Mobile-first، responsive design، SPAs، HTML5 (2014) وبعده HTML Living Standard (2019)
2020s	HTTP/3 (2022)، ويب بالـ AI، مفاهيم Web3، serverless

Internet vs World Wide Web 1.8

	Internet	World Wide Web
هو إيه	شبكة الشبكات (البنية التحتية)	مجموعة صفحات ويب — **خدمة فوق** الإنترنت
Protocol	TCP/IP	HTTP/HTTPS
اتعمل إمتى	الستينات (ARPANET)	1989 (Tim Berners-Lee)
بيشمل	Email, FTP, VoIP, Web	صفحات HTML و CSS و JS
بتوصله بـ	أي network protocol	المتصفحات

> مثال: الإيميل (SMTP) خدمة **إنترنت** مش جزء من الويب. Gmail مجرد واجهة ويب ليه.

1.9 مكونات أي صفحة ويب

Content → Structure (HTML) → Style (CSS) → Behavior (JavaScript)

> 💡 HTML هو الهيكل العظمي، CSS هو الجلد والهدوم، JavaScript هو الحركة.

1.10 من الـ URL لحد ما الصفحة تظهر (From URL to Pixels) ⭐

لما تكتب `https://example.com/` :

#	الخطوة	بيحصل إيه
1	**DNS lookup**	المتصفح يسأل DNS Resolver عن IP بتاع `example.com` ← يرجع `203.0.113.10`
2	**TCP handshake**	فتح اتصال موثوق بـ 3 رسايل: `SYN` وبعدها `SYN-ACK` وبعدها `ACK`
3	**TLS handshake**	التأكد من الـ certificate وعمل session keys للتشفير
4	**HTTP GET /**	المتصفح يطلب الصفحة ← السيرفر يرد `200 OK` + HTML document
5	**Parse HTML → DOM**	المتصفح يقرا الـ HTML ويبني الـ DOM، ويطلب ملفات CSS و JS والصور
6	**Style → Layout → Paint**	يحسب الشكل والأماكن ويرسم الصفحة على الشاشة

- المتصفح بيعمل **cache** لردود الـ DNS وبيعيد استخدام الاتصالات، فالطلبات اللي بعد كده بتعدّي خطوات 1–3.
- احفظ الترتيب: **DNS → TCP → TLS → HTTP → Render**

1.11 الـ HTTP Request & Response

كل رسالة HTTP فيها: **method** (زي `GET`)، **status code** (زي `200`)، و **headers** (زي `Content-Type`).

مثال توضيحي:

```
GET /index.html HTTP/1.1        ← method + path
Host: example.com               ← request header

HTTP/1.1 200 OK                 ← status code
Content-Type: text/html         ← response header
```

الـ Status codes — أول رقم بيحكيلك القصة:

الفئة	المعنى	أمثلة
2xx	Success نجاح	`200 OK`، `201 Created`، `204 No Content`
3xx	Redirection تحويل	`301 Moved Permanently`، `304 Not Modified`
4xx	Client error غلطة العميل	`400 Bad Request`، `403 Forbidden`، `404 Not Found`
5xx	Server error غلطة السيرفر	`500 Internal Server Error`، `503 Service Unavailable`

1.12 HTTP vs HTTPS

	HTTP	HTTPS
يعني	HyperText Transfer Protocol	HTTP **جوه اتصال مشفّر بـ TLS**
Default port	**80**	**443**

	HTTP	HTTPS
البيانات	plain text (أي حد في النص يقدر يقراها)	مشفرة
بيحمي	—	**Confidentiality** (محدش يتصنت)، **Integrity** (محدش يعدّل)، **Authenticity** (إنت بتكلم السيرفر الحقيقي)
المتصفح	بيكتب "Not Secure"	هو الستاندرد لكل المواقع دلوقتي

- **TLS** حل محل **SSL** القديم اللي بقى مش آمن — بس الناس لسه بتقول "SSL certificate".
- ميزات زي الـ geolocation والـ service workers والكاميرا **مبتشتغلش غير على HTTPS**.
- ⚠️ HTTPS بيثبت **إنت بتكلم مين** — مش إن الموقع نفسه أمين.

1.13 Developer Tools (DevTools)

تفتحها بـ **F12** أو `Ctrl+Shift+I` (Windows/Linux) أو `Cmd+Opt+I` (Mac) أو كليك يمين ← **Inspect**.

الـ Panel	بيعمل إيه
Elements	تشوف وتعدّل الـ HTML/CSS لايف
Console	تشغّل JavaScript وتشوف الـ errors والـ logs
Network	تشوف كل HTTP request: الـ method والـ status code والـ headers والوقت
Sources	تعمل debug لكود JavaScript
Lighthouse	تعمل audit للـ performance والـ accessibility والـ SEO

1.14 بيئة الشغل (Development Setup)

الأداة	الاستخدام
VS Code + Emmet	محرر مجاني؛ اكتب `!` وبعدين `Tab` يطلعلك هيكل HTML كامل
Live Server / Five Server	إضافة بتعمل reload للمتصفح أوتوماتيك لما تعمل save
Chrome / Firefox / Edge	المتصفح + DevTools
validator.w3.org	بيتأكد إن الـ HTML بتاعك صح
Git + GitHub	version control ونشر الشغل
GitHub Pages	استضافة مجانية: من Settings ادخل Pages واختار Deploy from branch `main`، والموقع يبقى على `https://<you>.github.io/<repo>/`

```
git clone https://github.com/<you>/web-programming.git   # تنزّل الريبو عندك
git add index.html                                       # تجهّز الملف للـ commit
git commit -m "Add home page"                            # تحفظ نسخة برسالة
git push                                                 # ترفع على GitHub
```

Try It 1.15: تشريح URL بالـ JavaScript

```
const url = new URL('https://example.com:8080/docs/intro.html?lang=en&page=2#setup');
console.log('protocol:', url.protocol);              // https:
console.log('host:', url.host);                     // example.com:8080
console.log('path:', url.pathname);                 // /docs/intro.html
console.log('query lang =', url.searchParams.get('lang')); // en
console.log('fragment:', url.hash);                 // #setup
```

1.16 تطور الويب

	Web 1.0 (1991–2004)	Web 2.0 (2004–دلوقتي)	Web 3.0 (لسه بيظهر)
المحتوى	صفحات static	محتوى من اليوزرز	Decentralized & semantic
التفاعل	Read-only	Read & Write	Read, Write & **Own**
التكنولوجيا	HTML, CSS	AJAX, JS Frameworks, APIs	Blockchain, AI, dApps
أمثلة	صفحات شخصية، directories	Facebook, YouTube, Wikipedia	Ethereum, IPFS, NFTs
مين متحكم في الداتا	صاحب الموقع	شركات المنصات	اليوزرز (wallets & tokens)

> "Web 3.0" لسه بيتناقش: الكلمة بتتقال على **Semantic Web** بتاع Tim Berners-Lee، وكمان على الملكية بالـ blockchain ("Web3").

1.17 أسئلة الكويز (بالإجابات)

السؤال	الإجابة
فتحت لينك وجالك `404` — غلطة مين؟ وَ `503` ؟	`404` = **4xx client error** (الريسورس مش موجود). `503` = **5xx server error** (السيرفر واقع أو عليه ضغط)
الإيميل جزء من الـ World Wide Web؟	**لأ** — الإيميل (SMTP) خدمة إنترنت. Gmail مجرد واجهة ويب ليه
رتّب: TLS handshake, render, DNS lookup, HTTP GET, TCP handshake	**DNS lookup** → **TCP handshake** → **TLS handshake** → **HTTP GET** → **render**
مين بيصون ستاندرد HTML النهارده؟	**WHATWG** (HTML Living Standard) بالاتفاق مع W3C

1.18 الخلاصة

- الإنترنت شبكة عالمية، والويب شغال فوقها.
- الويب بيستخدم Client-Server model مع HTTP/HTTPS.
- TCP/IP هو إطار الطبقات للاتصال.
- DNS بيحوّل أسماء الدومينات لـ IP.
- تحميل الصفحة: DNS → TCP → TLS → HTTP → render.

- رسالة HTTP فيها method و status code (2xx/3xx/4xx/5xx) و headers.
- IETF (بروتوكولات)، W3C و WHATWG (ستاندردات الويب)، ICANN (أسماء وأرقام).
- الموقع = HTML (هيكل) + CSS (شكل) + JavaScript (سلوك).
- أدواتك: VS Code + Live Server، DevTools، Git/GitHub Pages.

المحاضرة 02 — HTML5 Fundamentals

2.1 HTML يعني إيه؟

- **H**yper**T**ext **M**arkup **L**anguage
- بيوصف **المحتوى والهيكل** بتاع الصفحة — **مش الشكل** (الشكل شغلة CSS!)
- بيستخدم **tags** عشان يعلّم على المحتوى، فالمتصفح يعرف كل حتة دي إيه.
- الستاندرد الحالي: (WHATWG) **HTML Living Standard** — والكل لسه بيسميه HTML5.
- أحسن مرجع: **MDN Web Docs** — `developer.mozilla.org/en-US/docs/Web/HTML`

مصطلحات لازم تفرّق بينها:

```
<p class="intro">Welcome!</p>
```

- **opening tag** = `<p class="intro">` — وجواه **attribute** اسمه `class` وقيمته `"intro"`
- **content** = `Welcome!`
- **closing tag** = `</p>`
- الحتة كلها من أولها لآخرها = **element**

2.2 الهيكل الأساسي لأي صفحة HTML

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My First Page</title>
  </head>
  <body>
    <h1>Hello, World!</h1>
    <p>This is my first web page.</p>
  </body>
</html>
```

السطر	بيعمل إيه
`<!DOCTYPE html>`	بيقول للمتصفح استخدم **standards mode** (مش "quirks mode" القديم)
`<html lang="en">`	الـ **root element** (أبو الصفحة كلها). الـ `lang` بيعرّف الـ screen readers وأدوات الترجمة لغة الصفحة (للعربي: `lang="ar"`)
`<head>`	معلومات **عن** الصفحة — مش بتظهر جواها
`<meta charset="UTF-8">`	الـ encoding بتاع النص — **لازم يبقى أول حاجة في** `<head>`
`<meta name="viewport" ...>`	بيخلي الصفحة تظبط على شاشة الموبايل بدل ما تظهر نسخة desktop متصغّرة

السطر	بيعمل إيه
`<title>`	**إجباري** — بيظهر في التاب وفي نتايج البحث
`<body>`	كل المحتوى اللي بيظهر في الصفحة

⚡ **Shortcut:** في VS Code اكتب `!` واضغط `Tab` — الـ Emmet بيكتبلك الهيكل ده. التاجات مش case-sensitive بس المتعارف عليه إنها تتكتب **lowercase**.

2.3 الـ Attributes

```
<element attribute="value" attribute="value">content</element>
```

- بتدّي **معلومات زيادة** عن الـ element.
- **دايماً بتتكتب في الـ opening tag** في صورة `name="value"` (حط القيمة بين علامات تنصيص).
- أشهرهم: `href`، `src`، `alt`، `class`، `id`، `lang`.
- **Boolean attributes** ملهاش قيمة — مجرد وجودها يعني "true": `<input required>`، `<video controls>`.

```
<a href="about.html">About Us</a>
<img src="logo.png" alt="Company Logo">
<p class="intro" id="main-intro">Welcome!</p>
```

elements — **Void Elements** ملهاش محتوى ولا closing tag: `<br>`، `<hr>`، `<img>`، `<input>`، `<meta>`، `<link>`

الـ `/` في الآخر (`<br />`) عادة من XHTML — مسموحة بس الـ HTML بيتجاهلها. في الكورس بنكتب `<br>`.

2.4 الـ Title والـ Favicon

```
<head>
  <title>Chapter 2: HTML Basics</title>
  <link rel="icon" href="favicon.svg" type="image/svg+xml">
</head>
```

- `<title>` بيتحط جوه `<head>` — بيظهر في **التاب** والـ **bookmarks** مش جوه الصفحة.
- خلي الـ titles **فريدة ووصفية**: `"Contact - Cairo Bakery"` مش `"Page 1"`.
- الـ **Favicon** = الأيقونة الصغيرة اللي في التاب والـ bookmarks والـ history. استخدم `rel="icon"` — الـ `rel="shortcut icon"` القديم **obsolete**.

2.5 الـ Paragraphs والـ Headings

```
<p>This is a paragraph of text.</p>

<h1>Main Title</h1>
<h2>Section Title</h2>
<h3>Subsection</h3>
```

- `<p>` = فقرة. `<h1>` لحد `<h6>` = 6 مستويات عناوين (h1 الأكبر، h6 الأصغر).
- الاتنين **block-level**، والمتصفح بيحط مسافة فوقهم وتحتهم أوتوماتيك.
- استخدم العناوين عشان تعمل **outline منطقي** للصفحة — **متنطّش مستويات** (مثلاً تنط من h1 لـ h3 من غير h2 = غلط).
- اختار العنوان حسب **مستواه في الـ outline مش حجمه** — الحجم شغلة CSS.
- قاعدة: `<h1>` **واحد بس** في الصفحة.

2.6 اللينكات `<a>` (Anchors)

```
<p>Search for it on <a href="https://www.google.com/">Google</a>!</p>
<p>Read the <a href="#rules">course rules</a> below.</p>
<p>Email <a href="mailto:me@example.com">the instructor</a>.</p>
```

الـ `href` (= **H**ypertext **REF**erence) ممكن يشاور على:

نوع الـ href	مثال	بيعمل إيه
موقع تاني (absolute)	`https://www.google.com/`	يروح لموقع بره
صفحة في موقعك (relative)	`about.html`	يروح لصفحة جنبك
حتة في نفس الصفحة (fragment)	`#rules`	ينزل للعنصر اللي `id="rules"`
إيميل	`mailto:me@example.com`	يفتح برنامج الإيميل
تليفون	`tel:+201000000000`	يتصل (على الموبايل)

- الـ `<a>` عادة **inline** جوه النص، بس في HTML5 ممكن يلف **card كاملة** من block content.
- اكتب نص لينك مفهوم لوحده: **"course rules"** مش **"click here"**.

2.7 Relative vs Absolute Paths

تخيل إن موقعك كده:

```
site/
├── index.html
├── contact.html
├── img/
│   └── cat.jpg
└── blog/
    └── post.html
```

النوع	الكتابة	معناها	مثال (من `blog/post.html`)
Relative — نفس الفولدر	`"about.html"`	ملف جنبي	`"other-post.html"`
Relative — فولدر جوه	`"img/photo.jpg"`	ادخل فولدر `img`	—
Relative — فولدر فوق	`"../index.html"`	اطلع فولدر لفوق	`"../img/cat.jpg"` ✅
Relative — من الـ root	`"/img/photo.jpg"`	من أول الموقع	⚠️ على GitHub Pages project sites الـ root هو `username.github.io` مش الريبو بتاعك
Absolute	`"https://www.google.com/some-image.png"`	URL كامل بالـ protocol والدومين	للحاجات اللي بره موقعك

✅ **Best practice:** relative لملفات موقعك، absolute للحاجات الخارجية. المسارات على السيرفر **case-sensitive**: `Logo.png` ≠ `logo.png` .

2.8 الصور `<img>`

```
<img src="img/html5.jpg" alt="HTML5 Logo" width="400" height="400">

<!-- صورة كلينك -->
<a href="https://example.com">
  <img src="logo.png" alt="Example.com home">
</a>
```

Attribute	بيعمل إيه
`src`	مسار/URL الصورة
`alt`	وصف الصورة (accessibility) — **إجباري**
`width` / `height`	المقاس بالبكسل (**من غير** `px`) — بيخلي المتصفح يحجز مكان الصورة قبل ما تحمل، فالنص اللي تحتها ميتنططش

- `<img>` عنصر **inline** و **void**.
- لو الصورة جوه `<a>` بتبقى clickable والـ `alt` بتاعها بيبقى هو **نص اللينك**.
- متعتمدش على `title` (الـ tooltip) — اليوزر اللي على تاتش أو كيبورد عمره ما هيشوفه.

ليه الـ `alt` مهم؟

- **Accessibility**: الـ screen readers بتقراه للمكفوفين.
- بيظهر **لو الصورة فشلت تحمل**.
- بيساعد محركات البحث تفهم الصورة (**SEO**).
- **إجباري** — والصور الديكور بس تاخد `alt=""` (فاضي بس موجود) عشان الـ screen reader يعدّيها.
- اوصف **الغرض** مش اسم الملف: `alt="Sales rose 15% in Q3"` مش `alt="chart.png"` .

2.9 الصور الـ Responsive (`srcset` ، `sizes` ، `<picture>` ، `loading="lazy"`)

ℹ️ دي مذكورة في الأهداف والملخص والتكليف بس **ملهاش سلايد شرح**، فده شرح مختصر عشان تقدر تعمل التكليف.

```
<!-- نفس الصورة بأحجام مختلفة، والمتصفح يختار الأنسب -->
<img src="photo-800.jpg"
     srcset="photo-400.jpg 400w, photo-800.jpg 800w, photo-1600.jpg 1600w"
     sizes="(max-width: 600px) 100vw, 50vw"
     alt="Cairo tower at night" width="800" height="600"
     loading="lazy">

<!-- صورة مختلفة خالص حسب الشاشة —→
<picture>
  <source media="(min-width: 800px)" srcset="banner-wide.jpg">
  <img src="banner-narrow.jpg" alt="Our bakery" width="400" height="300">
</picture>
```

الحاجة	بتعمل إيه
`srcset`	قايمة نسخ من **نفس** الصورة بأحجام مختلفة (`400w` = عرض الصورة الحقيقي 400px)
`sizes`	بيقول للمتصفح الصورة هتتعرض بعرض قد إيه (هنا: 100% من الشاشة لو عرض الشاشة 600px أو أقل، غير كده نصها) — عشان يختار أنسب نسخة من الـ srcset
`<picture>` + `<source>`	لما تكون عايز صورة **مختلفة** (crop تاني أو format تاني) حسب الشاشة. الـ `<img>` جواها **إجباري** وهو الـ fallback
`loading="lazy"`	الصورة متتحملش غير لما اليوزر يقرّب منها بالـ scroll — للصور اللي تحت في الصفحة، **مش** الصور اللي فوق أول ما الصفحة تفتح

2.10 القوايم (Lists)

Ordered list `<ol>` — الترتيب مهم (خطوات، ترتيب)

```
<ol>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
<!— 1. 2. 3. —→

<ol start="5" reversed>
  <li>First item</li>
  <li>Second item</li>
  <li>Third item</li>
</ol>
<!— 5. 4. 3. —→
```

- `start="5"` = ابدأ العد من 5. `reversed` = عِد بالعكس.

- عايز حروف أو أرقام رومانية؟ دي شغلة CSS: `list-style-type`.

Unordered list `<ul>` — الترتيب مش مهم (مكونات، features، قوايم navigation)

```
<ul>
  <li>HTML</li>
  <li>CSS</li>
  <li>JavaScript</li>
</ul>
```

- `<ul>` و `<ol>` **مينفعش يكون جواهم غير** `<li>`. شكل الـ bullet بيتغير بالـ CSS.

Description list `<dl>` — مصطلح ووصفه (glossary، FAQ، key–value)

```
<dl>
  <dt>HTML</dt>
  <dd>HyperText Markup Language</dd>
  <dt>CSS</dt>
  <dd>Cascading Style Sheets</dd>
</dl>
```

- `<dl>` = الحاوية، `<dt>` = المصطلح (term)، `<dd>` = الوصف بتاع المصطلح اللي قبله.

Nested lists — قايمة جوه قايمة

```
<ul>
  <li>Web Languages
    <ul>
      <li>HTML</li>
      <li>CSS</li>
    </ul>
  </li>          <!-- لـ <li/> هنا بعد لـ <ul/> الداخلية -->
  <li>Server Languages</li>
</ul>
```

⚠️ الـ `<ul>` الداخلية بتتحط **جوه الـ** `<li>` **الأب، قبل** `</li>` **بتاعه** — عمرها ما تتحط مباشرة جوه الـ `<ul>` الخارجية.

2.11 الـ Character Entities

طريقة تكتب بيها حروف خاصة أو محجوزة في HTML. **بتبدأ بـ** `&` **وبتخلص بـ** `;`.

الحرف	الـ Entity	ملاحظة
<	`<`	**less than** — لازم تعمله escape
>	`>`	**greater than**
"	`"`	لازم جوه قيم الـ attributes

الحرف	الـ Entity	ملاحظة
&	`&`	**لازم** تعمله escape
©	`©`	copyright
™	`™`	trademark
é	`é`	
ñ	`ñ`	
π	`π`	
Δ	`Δ`	
(مسافة مبتتكسرش)	` `	non-breaking space: `10 km` تفضل على سطر واحد

- عشان تعرض `<p>` كنص: اكتب `<p>` .
- طالما عندك `<meta charset="UTF-8">` تقدر تكتب ©، π، é عادي — **لازم بس تعمل escape لـ** `<` **و** `&` (و `"` جوه قيم الـ attributes).
- **Challenge:** عايز تعرض النص `&` نفسه؟ اكتب `&` (بتعمل escape للـ `&` نفسها).

Block vs Inline Elements 2.12

	Block Elements	Inline Elements
العرض	بياخد العرض المتاح كله	بياخد قد محتواه بس
سطر جديد	بيبدأ على سطر جديد	مش بيبدأ سطر جديد — بيمشي مع النص
Margins	بعضهم ليه margin افتراضي (`<p>`، `<h1>`، `<ul>`) — الـ `<div>` ملوش	—
أمثلة	`<p>`، `<h1>` – `<h6>`، `<div>`، `<ul>`، `<ol>`، `<table>`، `<blockquote>`	`<a>`، `<em>`، `<strong>`، `<img>`، `<code>`، `<span>`

```
<em>text</em>
<em>text</em>
<em>text</em>
<p>text</p>
<p>text</p>
```

النتيجة: الـ 3 `<em>` على **نفس السطر** (`text text text`) — والـ newlines اللي بينهم في الكود بقت مسافة واحدة. وكل `<p>` على **سطر لوحده**.

> "Block" و "inline" هما في الحقيقة قيم `display` الافتراضية في CSS — والـ CSS يقدر يغيرهم. الـ HTML spec نفسه بيسميهم **flow content** و **phrasing content**.

2.13 قواعد الـ Nesting (التداخل)

- الـ block elements عموماً تقدر تحتوي block و inline: `<div>` ، `<li>` ، `<body>` ، `<form>` .
- ⚠️ `<p>` **يحتوي inline elements ونص بس** — لو حطيت `<div>` أو `<ul>` جوه `<p>` المتصفح بيقفل الـ paragraph في صمت.
- `<ul>` / `<ol>` جواهم `<li>` بس (بس ممكن يكون جواه nested list).
- `<a>` **مينفعش** يكون جواه تاني أو `<button>` .
- **مرة واحدة بس في الصفحة:** `<html>` ، `<head>` ، `<body>` — و `<main>` واحد ظاهر بس.
- في شك؟ استخدم **W3C Validator**: `validator.w3.org`

2.14 الجداول (Tables)

الهيكل الأساسي

```
<table>
  <tr>
    <th scope="col">Name</th>
    <th scope="col">Age</th>
  </tr>
  <tr>
    <td>Alice</td>
    <td>25</td>
  </tr>
</table>
```

التاج	بيعمل إيه
`<table>`	الجدول كله
`<tr>`	table **r**ow — صف
`<th>`	table **h**eader — خلية عنوان (bold و centered افتراضياً). الـ `scope="col"` أو `"row"` بيقول للـ screen reader الخلية دي عنوان لأنهي خلايا
`<td>`	table **d**ata — خلية بيانات

⚠️ الجداول للبيانات **الجدولية بس** — **عمرها** ما تستخدم لعمل layout للصفحة (ده شغل CSS Grid/Flexbox).

الأقسام والعنوان (Sections & Caption)

```
<table>
  <caption>Fruit Prices</caption>
  <thead>
    <tr><th>Item</th><th>Price</th></tr>
  </thead>
  <tbody>
    <tr><td>Apple</td><td>$1.00</td></tr>
    <tr><td>Banana</td><td>$0.50</td></tr>
  </tbody>
  <tfoot>
    <tr><td>Total</td><td>$1.50</td></tr>
  </tfoot>
</table>
```

`<caption>` = عنوان الجدول · `<thead>` = صفوف الهيدر · `<tbody>` = صفوف البيانات · `<tfoot>` = صفوف الفوتر (زي الـ Total).

دمج الخلايا: `colspan` و `rowspan`

```
<table>
  <tr>
    <th colspan="2">Full Name</th>   <!-- واخدة عمودين ←
    <th>Age</th>
  </tr>
  <tr>
    <td>First</td>
    <td>Last</td>
    <td rowspan="2">25</td>          <!-- واخدة صفين ←
  </tr>
  <tr>
    <td>Alice</td>
    <td>Smith</td>                  <!-- خليتين بس، لأن الـ 25 نازلة من فوق ←
  </tr>
</table>
```

- `colspan="2"` = الخلية تمتد **أفقياً** على عمودين.
- `rowspan="2"` = الخلية تمتد **رأسياً** على صفين ← الصف اللي بعدها بيبقى فيه خلية أقل.
- **القاعدة الذهبية:** عِد الخلايا في كل صف (مع حساب الـ spans) — كل صف لازم "يملا" نفس عدد الأعمدة.
- الـ borders بتيجي من CSS (`border` ، `border-collapse`) — الـ `border="1"` القديم **obsolete**.

حل تمرين الـ Timetable

```
<table>
  <caption>CET214 Weekly Schedule</caption>
  <thead>
    <tr><th scope="col">Day</th><th scope="col">Time</th><th scope="col">Session</th></tr>
  </thead>
  <tbody>
    <tr><th scope="row" rowspan="2">Sunday</th><td>10:00</td><td>Lecture</td></tr>
    <tr><td>12:00</td><td>Lab</td></tr>
    <tr><th scope="row">Tuesday</th><td colspan="2">Office hours</td></tr>
  </tbody>
</table>
```

2.15 تنسيق النص (Text Formatting)

التاج	المعنى	الشكل الافتراضي
`<strong>`	**أهمية** (importance)	**bold**
`<em>`	**تأكيد في النطق** بيغيّر المعنى (stress emphasis)	*italic*
`<b>`	نص شكله مختلف **من غير** أهمية زيادة (keywords)	bold
`<i>`	نص شكله مختلف (كلمات أجنبية، أسماء سفن)	italic
`<mark>`	نص متعلّم عليه (highlighted)	خلفية صفرا
`<del>`	نص اتشال	~~متشطب~~
`<ins>`	نص اتضاف	تحته خط
`<sub>`	subscript (زي H_2O)	تحت السطر
`<sup>`	superscript (زي x^2)	فوق السطر
`<code>`	كود	خط monospace

Semantic vs Presentational: `<strong>` و `<em>` ليهم **معنى**. `<b>` و `<i>` لسه موجودين للنص اللي شكله مختلف من غير أهمية زيادة. ولو عايز **شكل بس** ← استخدم CSS.

2.16 المسافات وكسر السطور (Whitespace)

الـ HTML **بيدمج المسافات**: كذا space أو newline بيبقوا مسافة واحدة.

الحاجة	بتعمل إيه
` `	بيكسر السطر جوه الفقرة
`<hr>`	خط أفقي (thematic break — فاصل موضوع)
`<pre>`	بيحافظ على المسافات والسطور **زي ما هي بالظبط**

الحاجة	بتعمل إيه
` `	مسافة مبتتدمجش ومبتتكسرش: `10 km`

```
<p>Roses are red,<br>Violets are blue,<br>HTML is fun,<br>And so are you!</p>
<hr>
<pre>
  This text preserves
    all     spacing
</pre>
```

⚠️ **متعملش مسافات بالـ HTML:** عمرك ما تستخدم `<br><br>` أو صف ` ` عشان تبعد حاجات — دي شغلة CSS (`margin`، `gap`).

2.17 أخطاء شائعة

الغلط	الصح
`<p>Hello World` (مفيش closing tag)	`<p>Hello World</p>`
`<strong><em>Bold and italic</strong></em>` (تداخل)	`<strong><em>…</em></strong>` — اقفل **بالعكس**
`<img src="photo.jpg">` (مفيش alt)	`<img src="photo.jpg" alt="Our team at the 2025 hackathon">`
`<a>Click here</a>` (مفيش href + نص مش واضح)	`<a href="schedule.html">Course schedule</a>`

2.18 صفحة HTML5 كاملة

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1">
    <title>My Portfolio</title>
    <link rel="icon" href="favicon.svg" type="image/svg+xml">
  </head>
  <body>
    <h1>Welcome to My Portfolio</h1>
    <img src="me.jpg" alt="Portrait of Sara smiling" width="200" height="200">
    <p>I am learning <strong>HTML5</strong> in the <em>Web Programming</em> course.</p>
    <h2>My Skills</h2>
    <ul>
      <li>HTML</li>
      <li>CSS</li>
      <li>JavaScript</li>
    </ul>
    <p>Contact me at <a href="mailto:me@example.com">me@example.com</a></p>
  </body>
</html>
```

2.19 أسئلة الكويز (بالإجابات)

السؤال	الإجابة
صفحتك `site/blog/post.html` — اكتب الـ src لـ `site/img/cat.jpg`	`../img/cat.jpg` — اطلع فولدر، وادخل `img`
الـ alt بتاع صورة ديكور (فاصل) يبقى إيه؟	`alt=""` (فاضي بس موجود) — الـ screen reader يعدّيها
ليه نحط `width` و `height` ؟	بيدّوا المتصفح الـ aspect ratio فيحجز المكان والنص اللي تحت ميتنططش
عايز تعرض لوجو شركة	`<img>` (مع alt!)
عايز لينك لصفحة تانية	`<a href="...">`
قايمة بـ bullets	`<ul>` مع `<li>`
بيانات في صفوف وأعمدة	`<table>`
نص bold عشان مهم	`<strong>` (أهمية semantic)

2.20 الخلاصة

- HTML بيوصف هيكل ومحتوى الصفحة.
- كل صفحة محتاجة `<!DOCTYPE html>` ، `<html lang>` ، `<head>` (فيه charset و viewport و `<title>`) و `<body>` .
- الـ attributes بتدّي معلومات زيادة (`href` ، `src` ، `alt`).

- العناوين `<h1>` – `<h6>` بتعمل outline منطقي.
- اللينكات `<a>` والصور `<img>` (بـ alt و width و height) أساس المحتوى.
- الصور الـ responsive: `srcset` / `sizes` ، `<picture>` ، و `loading="lazy"` .
- 3 أنواع قوايم: ordered و unordered و description.
- Character entities للحروف المحجوزة (`<` ، `&` ، `©`).
- افهم الفرق بين block و inline.
- الجداول بتنظم البيانات في صفوف وأعمدة بأقسام semantic.

المحاضرة 03 — Semantic HTML, Accessibility & CSS Fundamentals

المحاضرة 4 أجزاء: **CSS** ← **Semantic Tags** ← **Metadata** ← **Forms**.

الجزء 1 — HTML Forms

3.1 أساسيات الـ Form

```
<form action="/submit" method="post">
  <label for="name">Name:</label>
  <input type="text" id="name" name="name">
  <button type="submit">Submit</button>
</form>
```

الحاجة	بتعمل إيه
`<form>`	بيعمل منطقة تفاعلية لجمع input من اليوزر
`action`	الـ URL اللي البيانات هتتبعتله
`method`	البيانات هتتبعت **إزاي** (`GET` أو `POST`)
`<label for="name">` ↔️ `<input id="name">`	بيربط النص بالـ control (لازم الـ `for` = الـ `id`)
`name`	**المفتاح** اللي بيتبعت للسيرفر: لو كتبت "Sara" هيتبعت `name=Sara`
`<button type="submit">` (أو `<input type="submit">`)	بيبعت الـ form

💡 فرّق بين `id` و `name`: الـ `id` عشان الـ `<label>` والـ CSS والـ JS. الـ `name` هو اللي **بيتبعت للسيرفر**. من غير `name` القيمة مش هتتبعت.

3.2 ⭐ GET vs POST

	GET	POST
البيانات فين	في الـ URL: `/search?q=html&page=2`	في الـ **body** الـ HTTP request
ظاهرة فين	الـ address bar والـ history والـ server logs	مش في الـ URL (بس ظاهرة في DevTools)
الحجم	محدود بطول الـ URL (كام KB)	كبير (السيرفر بيحدده)
Cache & bookmarks	ينفع cache و bookmark	لأ
المعنى	**قراءة** بيانات — آمن تكرره	**تغيير** بيانات — بيطلع تحذير لو عملت resubmit
امتى	البحث، الفلترة	Login، تسجيل، رفع ملفات

⚠️ **POST مش تشفير!** عمرك ما تبعت password بـ GET (هيبان في الـ URL والـ history والـ logs). بس POST لوحده برضه مش آمن — **HTTPS بس هو اللي بيشفّر**.

3.3 أنواع الـ Input

```
<form>
  <label>Text: <input type="text" name="nickname" value="Sara"></label>
  <label>Password: <input type="password" name="pwd"></label>
  <label>Email: <input type="email" name="email"></label>
  <label>Number: <input type="number" name="age" value="21" min="0" max="100"></label>
  <label><input type="checkbox" name="subscribe" checked> Subscribe</label>
  <label><input type="radio" name="plan" value="free"> Free</label>
  <label><input type="radio" name="plan" value="pro" checked> Pro</label>
  <button type="submit">Submit</button>
</form>
```

type=	بيعمل إيه	Attributes بتيجي معاه
`text`	سطر نص عادي (الافتراضي)	`value` ، `minlength` ، `maxlength` ، `pattern`
`password`	النص بيظهر نقط ••••	`autocomplete="current-password"` / `"new-password"`
`email`	إيميل + **بيتأكد من الفورمات أوتوماتيك**	`required`
`number`	أرقام بس + أسهم زيادة/نقصان	`min` ، `max`
`checkbox`	مربع صح — ينفع تختار كذا واحد	`checked`
`radio`	دايرة — **واحد بس** من المجموعة	**نفس الـ** `name` للمجموعة + `value` **مختلف** لكل واحد + `checked`
`submit`	زرار بيبعت الـ form	`value` (نص الزرار)
`date`	منتقي تاريخ	`min` ، `max`
`tel`	رقم تليفون (بيفتح كيبورد أرقام على الموبايل)	`autocomplete="tel"`
`url`	لينك + بيتأكد من الفورمات	
`range`	slider	`min` ، `max`
`color`	منتقي ألوان	
`file`	رفع ملف	
`search`	خانة بحث	

طريقتين تربط بيهم الـ `<label>` :

```
<!-- 1) explicit: for = id -->
<label for="email">Email</label>
<input id="email" type="email" name="email">

<!-- 2) implicit: الـ label لافّ الـ input -->
<label>Email <input type="email" name="email"></label>
```

3.4 الـ Textarea والـ Select

```
<label for="msg">Message:</label>
<textarea id="msg" name="message" rows="4" cols="30"></textarea>

<label for="country">Country:</label>
<select id="country" name="country">
  <option value="">--Select--</option>
  <option value="eg">Egypt</option>
  <option value="sa">Saudi Arabia</option>
  <option value="ae">UAE</option>
</select>
```

- `<textarea>` = نص متعدد السطور. `rows` = عدد السطور الظاهرة، `cols` = العرض بالحروف.
  - ⚠️ خلي `</textarea>` **على نفس السطر** — أي مسافة جواه بتبقى نص افتراضي.
- `<select>` = قايمة منسدلة، وكل اختيار `<option>`. الـ `value` هو اللي بيتبعت للسيرفر، والنص هو اللي اليوزر بيشوفه.
- 💡 لما تدوس على الـ `<label>` المربوط صح، الـ input بياخد focus.

3.5 التجميع والـ Inputs الأذكى

```
<fieldset>
  <legend>Shipping address</legend>
  <label for="street">Street</label>
  <input id="street" name="street" autocomplete="street-address">

  <label for="zip">Postal code</label>
  <input id="zip" name="zip" inputmode="numeric" autocomplete="postal-code">
</fieldset>
```

الحاجة	بتعمل إيه	القيم
`<fieldset>` + `<legend>`	بيجمّع controls مرتبطة ببعض. الـ screen reader بيقول الـ legend ("Shipping address, Street...") **أساسي لمجموعات الـ radio!**	—
`autocomplete`	بيخلي المتصفح يملا بيانات هو عارفها	`name`، `email`، `tel`، `street-address`، `postal-code`، `username`، `current-password`، `new-password`
`inputmode`	بيختار **كيبورد الموبايل — من غير** ما يغيّر الـ validation	`numeric`، `decimal`، `tel`، `email`

⚠️ استخدم `type="number"` للكميات الحقيقية بس. الرقم البريدي ورقم الكارت ورقم التليفون **نصوص** (ممكن يبدأوا بصفر، ومحدش بيجمعهم) ← استخدم `type="text"` + `inputmode="numeric"`.

3.6 الـ Validation المدمج (من غير JavaScript!)

Attribute	بيعمل إيه	مثال
`required`	لازم الخانة تتملي قبل الإرسال	`<input required>`
`pattern`	regex **القيمة كلها** لازم تطابقه	`pattern="[A-Za-z0-9]+"` = حروف أو أرقام، واحد أو أكتر
`min` / `max`	أقل وأكبر قيمة (للأرقام)	`min="18" max="120"`
`minlength` / `maxlength`	أقل وأكبر عدد حروف	`minlength="3" maxlength="20"`
`type="email"` / `type="url"`	بيتأكد من الفورمات أوتوماتيك	

```
<form>
  <label for="user">Username (3-20 letters or digits)</label>
  <input id="user" name="username" required minlength="3" maxlength="20"
         pattern="[A-Za-z0-9]+" autocomplete="username">

  <label for="mail">Email</label>
  <input id="mail" type="email" name="email" required placeholder="you@example.com">

  <label for="age">Age</label>
  <input id="age" type="number" name="age" min="18" max="120">

  <button type="submit">Register</button>
</form>
```

- ⚠️ الـ `placeholder` مجرد **تلميح مش label** — بيختفي أول ما اليوزر يكتب.
- ⚠️ **اعمل validation على السيرفر كمان** — الـ HTML validation سهل حد يتخطاه.

حل تمرين الـ Registration Form:

```
<form action="/register" method="post">
  <fieldset>
    <legend>Account</legend>
    <label for="username">Username</label>
    <input id="username" name="username" required minlength="3" maxlength="20"
autocomplete="username">
    <label for="email">Email</label>
    <input id="email" type="email" name="email" required autocomplete="email">
  </fieldset>

  <fieldset>
    <legend>Plan</legend>
    <label><input type="radio" name="plan" value="free" checked> Free</label>
    <label><input type="radio" name="plan" value="pro"> Pro</label>
  </fieldset>

  <label for="phone">Phone</label>
  <input id="phone" type="tel" name="phone" autocomplete="tel">

  <label for="about">About you</label>
  <textarea id="about" name="about" maxlength="200"></textarea>

  <button type="submit">Register</button>
</form>
```

الجزء 2 — Web Page Metadata

3.7 تاجات `<meta>`

بتدّي معلومات عن الصفحة للمتصفحات ومحركات البحث والسوشيال ميديا. بتتحط في `<head>` ومبتظهرش في الصفحة.

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Semantic HTML Guide - CET214</title>
  <meta name="description" content="Learn HTML5 semantic tags and accessibility">
</head>
```

الـ Meta	الكتابة	بيعمل إيه
Charset	`<meta charset="UTF-8">`	encoding النص — **لازم أول حاجة في** `<head>`
Viewport	`<meta name="viewport" content="width=device-width, initial-scale=1">`	بيظبط الـ layout على الموبايل
Description	`<meta name="description" content="...">`	ملخص قصير بيظهر غالباً في نتايج البحث
Robots	`<meta name="robots" content="noindex, nofollow">`	تعليمات للـ crawlers — محتاجه **بس لو عايز تمنع** (`noindex` = متظهرش الصفحة في البحث، `nofollow` = متمشيش ورا لينكاتها)

الـ Meta	الكتابة	بيعمل إيه
~~Keywords~~	`<meta name="keywords">`	❌ جوجل بيتجاهله من 2009 — متكتبوش

🔎 **SEO Tip:** أهم حاجتين: `<title>` فريد و `<meta name="description">`.

الجزء 3 — HTML5 Semantic Tags

3.8 يعني إيه Semantic؟

Semantic HTML = تستخدم تاجات بتوصف **معنى وغرض** المحتوى، مش شكله بس.

الفايدة:

- **Accessibility**: الـ screen readers بتفهم هيكل الصفحة.
- **SEO**: محركات البحث بتفهم المحتوى أحسن.
- **Maintainability**: الكود أوضح وأسهل في القراية.
- **Interoperability**: بيشتغل بنفس الطريقة على كل المتصفحات والأجهزة.

قبل HTML5 الناس كانت بتكتب `<div id="header">` و `<div id="nav">` ... ("div soup"). التاجات الـ semantic حلت محل ده.

3.9 هيكل الصفحة الـ Semantic

```
<html>
├── <head>  → meta, title, link
└── <body>
    ├── <header>  → navigation ،اسم الموقع ،اللوجو
    ├── <nav>     → لينكات المنيو الأساسية
    ├── <main>
    │   ├── <section>  → محتوى بموضوع واحد
    │   │   ├── <article>  → قصة / بوست
    │   │   └── <article>  → قصة / بوست
    │   └── <aside>    → sidebar
    └── <footer>  → لينكات ،copyright
```

```
<body>
  <header><!— logo, site title, navigation bar —></header>
  <nav><!— primary navigation links —></nav>
  <main>
    <section>
      <!— a thematic grouping of content —>
      <article><!— self-contained, independently distributable —></article>
    </section>
    <aside><!— sidebar: related links, ads, supplementary info —></aside>
  </main>
  <footer><!— copyright, contact info, secondary links —></footer>
</body>
```

3.10 مرجع التاجات الـ Semantic

التاج	بيعمل إيه
`<header>`	محتوى تمهيدي أو أدوات تنقل — للصفحة **أو** لـ section
`<nav>`	مجموعة لينكات navigation (**ينفع أكتر من واحد** في الصفحة)
`<main>`	المحتوى **الأساسي** للـ body (**واحد بس ظاهر** في الصفحة)
`<section>`	تجميع محتوى بموضوع واحد، غالباً ليه heading
`<article>`	محتوى **مستقل بذاته** ينفع يتنشر لوحده (بوست، خبر، كومنت)
`<aside>`	محتوى له علاقة جانبية باللي حواليه (sidebar، pull quotes)
`<footer>`	الفوتر: copyright، معلومات التواصل، لينكات
`<figure>` / `<figcaption>`	صورة (أو diagram/كود) + الكابشن بتاعها
`<time datetime="...">`	تاريخ/وقت بصيغة يفهمها الكمبيوتر: `<time datetime="2026-03-01">1 March 2026</time>`
`<address>`	معلومات التواصل
`<search>`	منطقة البحث

3.11 `<section>` vs `<article>` ⭐

	`<article>`	`<section>`
الغرض	محتوى **مستقل بذاته**	**تجميع** محتوى مرتبط بموضوع
ينفع لوحده؟	أيوه (مثلاً في RSS feed)	لأ، معتمد على اللي حواليه
أمثلة	بوست، خبر، بوست في فوروم، كومنت	فصل في كتاب، tab panel، مجموعة خانات form
Heading	المفروض يبقى ليه	المفروض يبقى ليه
Nesting	ممكن يحتوي `<section>`	ممكن يحتوي `<article>`

🧪 **اختبار سريع:** لو خدت الحتة دي وحطيتها في موقع تاني لوحدها، هتفضل مفهومة؟ لو أيوه يبقى `<article>`، ولو لأ يبقى `<section>`.

3.12 حل تمرين "Fix the Div Soup"

```
<!-- ❌ قبل: div soup ⟶
<div id="top">
  <div class="logo">Cairo Eats</div>
  <div class="menu"><a href="/">Home</a> <a href="/menu">Menu</a></div>
</div>
<div id="content">
  <div class="post">
    <div class="title">Best Koshari in Town</div>
    <div>Posted <span>1 March 2026</span></div>
  </div>
</div>
<div id="bottom">© 2026 Cairo Eats</div>

<!-- ✅ بعد: semantic ⟶
<header>
  <p>Cairo Eats</p>
  <nav><a href="/">Home</a> <a href="/menu">Menu</a></nav>
</header>
<main>
  <article>
    <h2>Best Koshari in Town</h2>
    <p>Posted <time datetime="2026-03-01">1 March 2026</time></p>
  </article>
</main>
<footer>© 2026 Cairo Eats</footer>
```

3.13 الـ Nesting الصح

الـ closing tag لازم يقفل آخر tag اتفتح (اللي يتفتح الأول يتقفل الآخر):

```
<!-- ✅ صح -->
<p>This is <em>really, <strong>REALLY</strong></em> fun!</p>

<!-- ❌ غلط -->
<p>This is <em>really, <strong>REALLY</em> lots of</strong> fun!</p>
```

⚠️ **Invalid but rendered:** المتصفح ممكن يعرض الكود الغلط بإنه "يخمّن" تصليح — وكل متصفح ممكن يخمّن بشكل مختلف، والـ screen readers بتاخد هيكل بايظ. اعمل validate دايماً!

كويز الـ Nesting — الأخطاء في الكود الأصلي:

السطر	الغلط
4	`<li>About` متقفلتش — الـ `</li>` بتاعتها مكانها **بعد** الـ `</ul>` الداخلية
7	`<li>Contact` ناقصها `</li>`
10	`<nav>` المفروض تبقى `</nav>` (ناقص الـ slash)
15	`<p>` متقفلتش

السطر	الغلط
19	`<em>` و `<strong>` متداخلين — اقفل `</strong>` قبل `</em>`

3.14 `<div>` و `<span>`

	`<div>`	`<span>`
النوع	حاوية عامة **block**	حاوية عامة **inline**
المعنى	ملوش معنى ولا شكل افتراضي	ملوش معنى ولا شكل افتراضي
الاستخدام	تجميع حاجات عشان الـ layout/styling	تعليم حتة نص جوه سطر عشان تعمله style

```
<div class="box">
  <h3>A div container</h3>
  <p>This is a paragraph with a <span class="highlight">highlighted span</span> inside it.</p>
</div>
```

✅ استخدم التاجات الـ semantic (`<section>` ، `<article>` ، `<nav>`) لما المحتوى يكون ليه معنى واضح. استخدم `<div>` و `<span>` بس كـ styling hooks — بـ **class** مش CSS inline style.

الجزء 4 — CSS Fundamentals

3.15 CSS يعني إيه؟

- **C**ascading **S**tyle **S**heets — بتتحكم المتصفح يعرض الـ HTML elements إزاي.
- بتوصف **الشكل والـ layout**.
- **HTML = المحتوى، CSS = العرض (presentation).**
- تقدر تخلي أي element يظهر بشكل مختلف عن الافتراضي.

💡 HTML هو الهيكل العظمي، و CSS هو الجلد والهدوم بتوع الصفحة.

3.16 الطريقة الوحشة (HTML بتاع التسعينات)

```
<p>
  <font face="Arial">Welcome to Greasy Joe's.</font>
  You will <b>never</b>, <i>ever</i>, <u>EVER</u> beat
  <font size="+4" color="red">OUR</font> prices!
</p>
```

- `<font>` **obsolete — اتشال من HTML5**، عمرك ما تستخدمه.
- **مشاكل الطريقة دي:**
  - **Accessibility**: الـ screen readers مبتفهمش الـ styling البصري.
  - **تنظيم الكود**: بتخلط المحتوى بالشكل.

- **الصيانة**: لو عايز تغيّر الشكل لازم تعدّل كل تاج لوحده.

- النهارده: `<b>` ، `<i>` ، `<u>` لسه موجودين بس ليهم **معنى** مش شكل — للأهمية/التأكيد استخدم `<strong>` / `<em>` ، وكل الشكل يبقى في CSS.

3.17 تلات طرق تحط بيهم CSS ⭐

1) Inline — الـ `style` attribute

```
<h2 style="font-family: Verdana, sans-serif;">Heading</h2>
```

2) Page-level (Internal) — تاج `<style>` في الـ `<head>`

```
<head>
  <style>
    h1 { font-size: x-large; font-weight: bold; }
    h2 { font-size: large;   font-weight: bold; }
  </style>
</head>
```

3) External — ملف `.css` منفصل بتربطه بـ `<link>`

```
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

	Inline (`style=""`)	Page-level (`<style>`)	External (`<link>`)
مكانها	جوه الـ tag نفسه	جوه `<head>` في نفس الصفحة	ملف `.css` لوحده
بتأثر على	element واحد بس	الصفحة دي بس	**الموقع كله** (كل صفحة تعمل link)
مميزات	سريعة للتجربة	rule واحدة تأثر على كل العناصر المطابقة	غيّر ملف واحد يتغير الموقع كله + المتصفح بيعمله **cache**
عيوب	**أقل طريقة كفاءة** — لازم تكررها على كل element. وبتغلب تقريباً أي rule في الـ stylesheet فصعب تعملها override	محصورة في صفحة واحدة	—
الحكم	❌ تجنبها	🆗	✅ **الستاندرد في المواقع الحقيقية**

- الـ declaration بتتكتب `property: value;` — وأكتر من declaration بينهم `;` (والـ `;` الأخيرة اختيارية بس يُفضّل تكتبها).
- مش محتاج `type="text/css"` على `<style>` — CSS هو الافتراضي في HTML5 (هتشوفه في كود قديم بس).
- في `<link>` : الـ `href` = مسار ملف الـ CSS، والـ `rel="stylesheet"` = نوع العلاقة.

تمرين "Three Ways, One Result": نفس الـ rule في التلات حالات `h2 { color: navy; font-weight: bold; }` — اللي بيتغير بس هو **مكانها**. ولو ربطت نفس `style.css` بصفحتين، الاتنين بيتغيروا مع بعض.

3.18 الـ CSS Syntax

```
selector {
  property: value;
  property: value;
}
```

```
p {                     /* style هيتعمله مين اللي :selector الـ */
  color: red;           /* ← declaration = property: value; */
  font-family: sans-serif;
}
```

- الـ **selector** بيشاور على الـ element(s) اللي عايز تعملها style.
- الـ **declaration block** جوه `{ }`.
- كل declaration فيها **property** و **value** بينهم `:`، والـ declarations بينها `;`.
- الكومنتات `/* ... */` بس — **مفيش** `//` **في CSS**.

3.19 الـ Selectors

```
/* في الصفحة <strong> كل الـ — Tag selector/*
strong { color: red; }

/* (بفاصلة) تاج لكذا ستايل نفس — Grouping/*
p, ol, ul { color: blue; }
/* = p { color: blue; }  ol { color: blue; }  ul { color: blue; } */

/* (CSS reset في الـ بيتستخدم) element كل — Universal */
* { margin: 0; padding: 0; }

/* إعدادات للصفحة كلها — Body/*
body { font-family: Arial, sans-serif; color: #333333; }

/* (بمسافة) بس <ol> اللي جوه <Descendant (contextual) — <em/*
ol em { color: blue; }

/* "class="warning عنده element أي — Class */
.warning { color: red; font-weight: bold; }

/* "id="important الوحيد اللي element الـ — ID */
#important { color: red; font-weight: bold; }

/* مع بعض class + descendant + تاج/*
ul li.important { color: red; }

/* واحدة rule في نوع كذا — Mixing/*
h1, #headline, .heading, .warning { font-size: large; color: green; }
```

> ⚠️ **الفاصلة vs المسافة — أهم فرق:**
>
> - `ol, em { }` (بفاصلة) = **Grouping**: كل `<ol>` **و** كل `<em>` كل واحد لوحده.
> - `ol em { }` (بمسافة) = **Descendant**: الـ `<em>` **اللي جوه** `<ol>` بس.

- التاجات الـ semantic بتريّحك: `nav a` ، `footer p` ، `article h2` من غير ما تحتاج classes.
- الـ CSS reset الحديث بيضيف كمان `box-sizing: border-box` (هتعرف ليه في محاضرة 04).

3.20 الـ Specificity — نظرة أولى

لما rule-ين يحطوا نفس الـ property على نفس الـ element، الـ selector الأكثر تحديداً (specific) هو اللي بيكسب.

```
cite    { color: green; }                    /* Rule 1 :كل الـ cite */
p cite  { font-style: italic; color: red; }  /* Rule 2 :أدق — cite جوه p */
li cite { font-weight: bold;  color: blue; } /* Rule 3 :أدق — cite جوه li */
```

الـ `<cite>` موجود فين	لونه	ليه
مش جوه `<p>` ولا `<li>`	green 🟢	Rule 1 بس اللي بتنطبق
جوه `<p>`	red 🔴	`p cite` بتسمّي **2** elements، و `cite` بتسمّي **1**، فـ Rule 2 أدق وبتكسب
جوه `<li>`	blue 🔵	Rule 3 بتغلب Rule 1 بنفس الطريقة

ترتيب القوة باختصار (هتتشرح بالتفصيل في محاضرة Cascade & Specificity):

من الأضعف للأقوى	الـ Selector	مثال
1 (أضعف)	Tag	`cite`
2	Descendant بتاجات أكتر	`p cite`
3	Class	`.warning`
4	ID	`#important`
5 (أقوى)	Inline style	`style="..."`

3.21 الـ Classes والـ IDs

```
<p class="warning">This is a warning.</p>
<p id="important">This is important.</p>
```

```
.warning   { color: red; font-weight: bold; }
#important { color: red; font-weight: bold; }
```

	Class	ID
في الـ HTML	`class="warning"` (من غير نقطة)	`id="important"` (من غير #)
في الـ CSS	`.warning` (**بنقطة**)	`#important` (**بـ hash**)

	Class	ID
التكرار	**كذا element** ينفع ياخدوا نفس الـ class	**element واحد بس** في الصفحة — لازم يكون فريد
كذا واحد على نفس الـ element؟	أيوه: `class="warning large"`	لأ، id واحد
استخدامات تانية	—	JavaScript (`getElementById`)، لينكات `<label for="...">`، `#fragment`
للـ styling	✅ **استخدم ده**	❌ specific أوي وصعب تعمله override — استخدمه كـ hook بس

3.22 التكليف العملي (Semantic Blog Page)

جزء 1 — `blog.html`:

- Head: `charset`، `viewport`، `<title>` فريد و `description`.
- `<header>` (عنوان المدونة + لوجو) · `<nav>` فيها list من 4 لينكات أو أكتر (Home | Blog | About | Contact).
- `<main>`: كذا `<article>` كل واحد فيه `h2` + `img` بـ `alt` وصفي + `p`. الصور الديكور `alt=""`.
- `<aside>`: خانة بحث `<input type="search">` ليها label + لينكات.
- `<footer>`: Contact form (label لكل خانة، `<fieldset>` / `<legend>`، `autocomplete`، + validation attributes) + copyright.
- **0 errors** في W3C Validator.

جزء 2 — `style.css`: (من غير أي inline style)

1. اربطه بـ `<link rel="stylesheet">`؛ ابدأ بـ `*` reset وإعدادات على `body`.
2. Tag و descendant selector + grouping selectors واحد على الأقل (زي `nav a`، `article h2`).
3. Classes اتنين على الأقل، و element عنده الاتنين؛ والـ ID بس كـ hook (زي target لينك داخلي).

اعمل validate للـ CSS على `jigsaw.w3.org/css-validator` وللـ HTML على `validator.w3.org`.

📋 الجدول الشامل لكل التاجات

المفتاح: V = Void · I = Inline · B = Block (ملوش closing tag) · H = في الـ head (مش ظاهر) · المحاضرة = اتشرح فين

هيكل الصفحة والـ Head

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<!DOCTYPE html>`	يشغّل الـ standards mode (مش تاج حقيقي، declaration)	—	—	02
`<html>`	الـ root element — بيلف الصفحة كلها	`lang`	—	02
`<head>`	معلومات عن الصفحة (مش ظاهرة)	—	H	02
`<meta>`	metadata: encoding، viewport، description، robots	`charset` ، `name` ، `content`	V, H	02, 03
`<title>`	عنوان الصفحة في التاب ونتايج البحث — **إجباري**	—	H	02
`<link>`	يربط ملف خارجي (CSS، favicon)	`rel` ، `href` ، `type`	V, H	02, 03
`<style>`	CSS جوه الصفحة نفسها	— (`type="text/css"` مش محتاجه)	H	03
`<body>`	كل المحتوى الظاهر	—	B	02

النصوص

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<h1>` – `<h6>`	عناوين 6 مستويات (h1 واحد بس في الصفحة، متنطّش مستويات)	—	B	02
`<p>`	فقرة (جواها inline ونص بس)	—	B	02
` `	كسر سطر جوه الفقرة	—	V, I	02
`<hr>`	خط أفقي / فاصل موضوع (thematic break)	—	V, B	02
`<pre>`	نص بيحافظ على المسافات والسطور زي ما هي	—	B	02
`<blockquote>`	اقتباس طويل من مصدر تاني	—	B	02
`<strong>`	أهمية (bold)	—	I	02
`<em>`	تأكيد في النطق بيغير المعنى (italic)	—	I	02
`<b>`	نص شكله مختلف من غير أهمية (keywords)	—	I	02, 03
`<i>`	نص شكله مختلف (كلمات أجنبية، أسماء سفن)	—	I	02, 03
`<u>`	نص ليه معنى/annotation (مش للتسطير الديكور)	—	I	03
`<mark>`	نص highlighted	—	I	02
`<del>`	نص اتشال (متشطب)	—	I	02
`<ins>`	نص اتضاف (تحته خط)	—	I	02

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<sub>`	subscript (H_2O)	—	I	02
`<sup>`	superscript (x^2)	—	I	02
`<code>`	كود (خط monospace)	—	I	02
`<cite>`	اسم عمل (كتاب، فيلم...) — اتستخدم في أمثلة الـ CSS	—	I	03
~~`<font>`~~	❌ **Obsolete** — اتشال من HTML5	~~`color`~~ ، ~~`size`~~ ، ~~`face`~~	I	03

اللينكات والصور والميديا

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<a>`	لينك (لصفحة، لحتة في الصفحة، mailto، tel)	`href`	I	02
`<img>`	صورة	`src` ، `alt` ، `width` ، `height` ، `srcset` ، `sizes` ، `loading`	V, I	02
`<picture>`	يلف `<source>` و `<img>` عشان صورة مختلفة حسب الشاشة	—	I	02
`<source>`	نسخة صورة جوه `<picture>` بشرط	`srcset` ، `media`	V	02 (شرح زيادة)
`<video>`	فيديو (اتذكر كمثال للـ boolean attribute)	`controls`	I	02
`<figure>`	صورة/محتوى مستقل + كابشن	—	B	03
`<figcaption>`	الكابشن بتاع الـ figure	—	B	03

القوايم

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<ol>`	قايمة مرقّمة (الترتيب مهم) — جواها `<li>` بس	`start` ، `reversed`	B	02
`<ul>`	قايمة bullets (الترتيب مش مهم) — جواها `<li>` بس	—	B	02
`<li>`	عنصر في القايمة (ممكن يحتوي nested list)	—	B	02
`<dl>`	description list (glossary، FAQ)	—	B	02
`<dt>`	المصطلح	—	B	02
`<dd>`	وصف المصطلح اللي قبله	—	B	02

الجداول

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<table>`	الجدول (للبيانات بس، مش layout)	obsolete ~~`border`~~	B	02
`<caption>`	عنوان الجدول	—	—	02
`<thead>`	يجمّع صفوف الهيدر	—	—	02

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<tbody>`	يجمّع صفوف البيانات	—	—	02
`<tfoot>`	يجمّع صفوف الفوتر	—	—	02
`<tr>`	صف	—	—	02
`<th>`	خلية عنوان (bold و centered)	`scope` ، `colspan` ، `rowspan`	—	02
`<td>`	خلية بيانات	`colspan` ، `rowspan`	—	02

الـ Forms

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<form>`	منطقة جمع بيانات من اليوزر	`action` ، `method`	B	03
`<label>`	نص بيوصف control — لما تدوس عليه يعمل focus	`for`	I	03
`<input>`	خانة إدخال (نوعها حسب `type`)	`type` ، `id` ، `name` ، `value` ، `checked` ، `required` ، `placeholder` ، `autocomplete` ، `inputmode` ، `maxlength` ، `minlength` ، `max` ، `min` ، `pattern`	V, I	03
`<button>`	زرار (`type="submit"` يبعت الفورم)	`type`	I	03
`<textarea>`	نص متعدد السطور	`id` ، `name` ، `rows` ، `cols` ، `maxlength`	I	03
`<select>`	قايمة منسدلة	`id` ، `name`	I	03
`<option>`	اختيار جوه الـ select	`value`	—	03
`<fieldset>`	يجمّع controls مرتبطة (أساسي للـ radio)	—	B	03
`<legend>`	عنوان الـ fieldset (الـ screen reader بيقوله)	—	—	03

الـ Semantic Layout

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<header>`	مقدمة/لوجو/navigation للصفحة أو لـ section	—	B	03
`<nav>`	مجموعة لينكات navigation (ينفع أكتر من واحدة)	—	B	03
`<main>`	المحتوى الأساسي (واحد بس ظاهر)	—	B	03
`<section>`	تجميع محتوى بموضوع واحد + heading	—	B	03
`<article>`	محتوى مستقل ينفع يتنشر لوحده	—	B	03
`<aside>`	محتوى جانبي (sidebar)	—	B	03
`<footer>`	فوتر: copyright، تواصل، لينكات	—	B	03
`<time>`	تاريخ/وقت مقروء للكمبيوتر	`datetime`	I	03

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<address>`	معلومات التواصل	—	B	03
`<search>`	منطقة البحث	—	B	03

الحاويات العامة (ملهاش معنى)

التاج	بيعمل إيه	أهم الـ Attributes	النوع	المحاضرة
`<div>`	حاوية عامة block — styling/layout hook	`class`، `id`	B	02, 03
`<span>`	حاوية عامة inline — لتعليم حتة نص	`class`، `id`	I	02, 03

📋 الجدول الشامل لكل الـ Attributes

Global Attributes (تتحط على أي تاج تقريباً)

Attribute	بيعمل إيه	مثال
`id`	اسم **فريد** للعنصر — لـ `#fragment` links، و `<label for>`، و JS، و CSS `#id`	`<p id="main-intro">`
`class`	اسم مجموعة (ينفع يتكرر، وينفع كذا class على عنصر) — للـ CSS `.class`	`<p class="warning large">`
`style`	CSS inline (❌ تجنبه)	`<h2 style="color: navy;">`
`lang`	لغة المحتوى (للـ screen readers والترجمة)	`<html lang="en">`
`title`	tooltip لما تقف بالماوس — ⚠️ متعتمدش عليه (التاتش والكيبورد مبيشوفوهوش)	`<img title="...">`

Head (`<meta>` و `<link>`)

Attribute	على أنهي تاج	بيعمل إيه	مثال
`charset`	`<meta>`	encoding النص — أول حاجة في الـ head	`<meta charset="UTF-8">`
`name`	`<meta>`	نوع الـ metadata	`name="viewport"`، `"description"`، `"robots"`
`content`	`<meta>`	قيمة الـ metadata	`content="width=device-width, initial-scale=1"`
`rel`	`<link>`	نوع العلاقة بالملف	`rel="stylesheet"`، `rel="icon"` (~~`shortcut icon`~~ obsolete)
`href`	`<link>`	مسار الملف	`href="style.css"`
`type`	`<link>`	نوع الملف (MIME)	`type="image/svg+xml"`

اللينكات والصور

Attribute	على أنهي تاج	بيعمل إيه	مثال
`href`	`<a>`	الوجهة: URL، ملف، `#id`، `mailto:`، `tel:`	`href="#rules"`
`src`	`<img>`	مسار الصورة	`src="img/logo.png"`
`alt`	`<img>`	وصف الصورة — **إجباري** (`""` للديكور)	`alt="HTML5 Logo"`
`width` / `height`	`<img>`	المقاس بالبكسل من غير `px` — يحجز المكان	`width="400" height="400"`
`srcset`	`<img>`، `<source>`	نسخ بأحجام مختلفة	`srcset="a-400.jpg 400w, a-800.jpg 800w"`
`sizes`	`<img>`	عرض الصورة على الشاشة عشان المتصفح يختار	`sizes="(max-width: 600px) 100vw, 50vw"`
`loading`	`<img>`	`lazy` = متحملش غير لما تقرب منها	`loading="lazy"`

Attribute	على أنهي تاج	بيعمل إيه	مثال
`media`	`<source>`	شرط الشاشة	`media="(min-width: 800px)"`
`controls`	`<video>`	boolean — يظهر أزرار التشغيل	`<video controls>`

القوايم والجداول

Attribute	على أنهي تاج	بيعمل إيه	مثال
`start`	`<ol>`	يبدأ العد من رقم معين	`start="5"`
`reversed`	`<ol>`	boolean — يعد بالعكس	`<ol reversed>`
`scope`	`<th>`	الخلية دي عنوان للعمود ولا للصف (للـ screen readers)	`scope="row"` / `scope="col"`
`colspan`	`<th>` ، `<td>`	الخلية تمتد على كذا **عمود**	`colspan="2"`
`rowspan`	`<th>` ، `<td>`	الخلية تمتد على كذا **صف**	`rowspan="2"`
~~`border`~~	~~`<table>`~~	❌ obsolete — استخدم CSS `border`	—

الـ Forms

Attribute	على أنهي تاج	بيعمل إيه	مثال
`action`	`<form>`	الـ URL اللي البيانات هتروحله	`action="/submit"`
`method`	`<form>`	`get` (في الـ URL) أو `post` (في الـ body)	`method="post"`
`for`	`<label>`	يربط الـ label بالـ input اللي `id` بتاعه نفس القيمة	`for="email"`
`type`	`<input>`	نوع الخانة	`text` ، `password` ، `email` ، `number` ، `checkbox` ، `radio` ، `date` ، `tel` ، `url` ، `range` ، `color` ، `file` ، `search` ، `submit`
`type`	`<button>`	نوع الزرار	`type="submit"`
`name`	`<input>` ، `<select>` ، `<textarea>`	**المفتاح اللي بيتبعت للسيرفر** (والـ radios بتاعة نفس المجموعة نفس الـ name)	`name="email"` ← `email=...`
`value`	`<input>` ، `<option>`	القيمة (الافتراضية أو اللي بتتبعت)	`value="pro"`
`checked`	checkbox / radio	boolean — متعلّم عليه من الأول	`<input type="checkbox" checked>`
`required`	`<input>` ، `<select>` ، `<textarea>`	boolean — لازم يتملى	`<input required>`

Attribute	على أنهي تاج	بيعمل إيه	مثال
`pattern`	`<input>`	regex القيمة كلها لازم تطابقه	`pattern="[A-Za-z0-9]+"`
`min` / `max`	number، date، range	أقل/أكبر **قيمة**	`min="18" max="120"`
`minlength` / `maxlength`	`<input>`، `<textarea>`	أقل/أكبر عدد **حروف**	`minlength="3" maxlength="20"`
`placeholder`	`<input>`، `<textarea>`	تلميح بيختفي لما تكتب — **مش label**	`placeholder="you@example.com"`
`autocomplete`	`<input>`	المتصفح يملا بيانات معروفة	`name`، `email`، `tel`، `username`، `street-address`، `postal-code`، `current-password`، `new-password`
`inputmode`	`<input>`	نوع كيبورد الموبايل (من غير ما يغير الـ validation)	`numeric`، `decimal`، `tel`، `email`
`rows` / `cols`	`<textarea>`	عدد السطور / العرض بالحروف	`rows="4" cols="30"`

Semantic

Attribute	على أنهي تاج	بيعمل إيه	مثال
`datetime`	`<time>`	التاريخ بصيغة يفهمها الكمبيوتر	`<time datetime="2026-03-01">1 March 2026</time>`

🎨 جداول CSS

الـ Selectors

الـ Selector	الكتابة	بيختار مين	مثال
Tag / Type	`p`	كل عناصر النوع ده	`strong { color: red; }`
Grouping	`a, b` (فاصلة)	كل `a` **و** كل `b`	`p, ol, ul { color: blue; }`
Universal	`*`	**كل** element (للـ reset)	`* { margin: 0; padding: 0; }`
Body	`body`	إعدادات للصفحة كلها	`body { font-family: Arial, sans-serif; }`
Descendant	`a b` (مسافة)	`b` اللي **جوه** `a` بس	`ol em { color: blue; }`
Class	`.name`	كل عنصر عنده `class="name"`	`.warning { color: red; }`
ID	`#name`	العنصر الوحيد اللي `id="name"`	`#important { color: red; }`
Tag + class	`tag.name`	عنصر من النوع ده **وعنده** الـ class	`ul li.important { color: red; }`
Mixing	`h1, #x, .y`	خليط في rule واحدة	`h1, #headline, .heading { color: green; }`

الـ Properties اللي ظهرت في السلايدز

Property	بتعمل إيه	قيم ظهرت
`color`	لون النص	`red` ، `blue` ، `navy` ، `green` ، `#333333`
`background`	الخلفية	`#e0f0ff`
`font-family`	نوع الخط (أكتر من واحد كـ backup)	`Verdana, sans-serif` ، `Arial, sans-serif`
`font-size`	حجم الخط	`x-large` ، `large`
`font-weight`	تخانة الخط	`bold` ، `normal`
`font-style`	ميل الخط	`italic` ، `normal`
`margin`	مسافة **بره** العنصر	`0`
`padding`	مسافة **جوه** العنصر	`0` ، `10px`
`border` / `border-collapse`	حدود الجدول	—
`list-style-type`	شكل ترقيم/bullet القايمة (حروف، روماني...)	—
`gap`	مسافة بين العناصر (بدل `  `)	—
`box-sizing`	جزء من الـ reset الحديث (محاضرة 04)	`border-box`

⚖️ حاجات بتتلخبط فيها

الحاجتين	الفرق في جملة
Internet vs **Web**	الإنترنت = البنية (TCP/IP). الويب = خدمة فوقها (HTTP + صفحات)
HTTP vs **HTTPS**	HTTPS = HTTP جوه TLS مشفّر. بورت 80 vs 443
4xx vs **5xx**	4xx غلطة الـ client (404). 5xx غلطة الـ server (500، 503)
GET vs **POST**	GET في الـ URL للقراءة/البحث. POST في الـ body للتغيير/Login. التشفير من HTTPS بس
Relative vs **Absolute**	Relative لملفات موقعك (`../img/a.jpg`). Absolute للخارج (`https://...`)
Block vs **Inline**	Block ياخد السطر كله ويبدأ سطر جديد. Inline يمشي مع النص
`<b>` vs `<strong>`	strong = مهم. b = شكله مختلف من غير أهمية
`<i>` vs `<em>`	em = تأكيد بيغيّر المعنى. i = كلمة أجنبية/اسم
`<dl>` vs `<ul>` vs `<ol>`	مرقّمة (الترتيب مهم) / bullets / مصطلح ووصفه
`<td>` vs `<th>`	خلية عنوان vs خلية بيانات
`rowspan` vs `colspan`	دمج أفقي (أعمدة) vs دمج رأسي (صفوف)
`<section>` vs `<article>`	article مستقل ينفع يتنشر لوحده. section تجميع بموضوع
`<span>` vs `<div>`	حاوية عامة inline vs block — الاتنين ملهمش معنى
`<div>` vs **semantic tags**	لو المحتوى ليه معنى (nav، article...) استخدم الـ semantic. الـ div للـ styling بس
`id` vs `name` (في الفورم)	id للـ label والـ CSS والـ JS. name هو اللي بيتبعت للسيرفر
`<label>` vs `placeholder`	label دايماً ظاهر ومربوط. placeholder تلميح بيختفي — **مش بديل**
`type` vs `inputmode`	type بيغيّر الـ validation والسلوك. inputmode بيغيّر الكيبورد بس
`min/max` vs `minlength/maxlength`	قيمة الرقم vs عدد الحروف
radio vs **checkbox**	radio اختيار واحد (نفس الـ name). checkbox كذا اختيار
External CSS vs **Internal** vs **Inline**	الموقع كله ✅ / صفحة واحدة / عنصر واحد
`p, em` vs `p em`	فاصلة = الاتنين لوحدهم. مسافة = em اللي جوه p بس
`.class` vs `#id`	class يتكرر ✅ للـ styling. id فريد، specific أوي
`margin` vs `padding`	margin بره العنصر. padding جوه العنصر

⚠️ Checklist قبل ما تسلّم أي صفحة

الهيكل

- [ ] `<!DOCTYPE html>` في أول سطر
- [ ] `<html lang="...">`
- [ ] `<meta charset="UTF-8">` **أول** حاجة في الـ head + `viewport` + `<title>` فريد (+ `description`)
- [ ] `<html>` و `<head>` و `<body>` مرة واحدة، و `<main>` واحد ظاهر

المحتوى

- [ ] `<h1>` واحد بس ومفيش مستويات عناوين متنطوطة
- [ ] كل `<img>` ليها `alt` (وصفي، أو `""` للديكور) + `width` و `height`
- [ ] كل `<a>` ليه `href` ونص واضح (مش "click here")
- [ ] كل `<li>` جوه `<ul>` أو `<ol>`، والـ nested list جوه الـ `<li>`
- [ ] مفيش `<div>` أو `<ul>` جوه `<p>`
- [ ] كل التاجات مقفولة وبالترتيب العكسي الصح (مفيش overlap)
- [ ] `<` و `&` في النص مكتوبين `<` و `&`
- [ ] الجداول للبيانات بس، وكل صف بيملا نفس عدد الأعمدة

الـ Forms

- [ ] كل control ليه `<label>` ظاهر ومربوط (`for` = `id`)
- [ ] كل control ليه `name`
- [ ] الـ radios بتاعة نفس المجموعة ليها نفس الـ `name` و `value` مختلف، وجوه `<fieldset>` + `<legend>`
- [ ] passwords بـ `method="post"` (وعلى HTTPS)
- [ ] `type="number"` للكميات بس (مش للتليفون والرقم البريدي)

الـ CSS

- [ ] مفيش inline `style` — كله في `style.css` مربوط بـ `<link rel="stylesheet">`
- [ ] مفيش `<font>` ولا `border="1"` ولا `<br><br>` للمسافات
- [ ] الـ styling بالـ classes، والـ ID كـ hook بس
- [ ] الكومنتات `/* */` مش `//`

الـ Validation

- [ ] HTML: `validator.w3.org` — 0 errors
- [ ] CSS: `jigsaw.w3.org/css-validator`